

Laboratório Prático: Modelos N-gram e a Intuição da Linguagem

Disciplina: Processamento de Linguagem Natural (PLN)

Ambiente: Google Colab / Jupyter Notebook

Objetivos

- Desmistificar a predição de texto mostrando que algoritmos aprendem padrões de repetição e não regras gramaticais estritas.
- Compreender o conceito fundamental de *N-grams* (Unigramas, Bigramas e Trigramas).
- Implementar, passo a passo, um algoritmo de "*autocompletar*" (previsão da próxima palavra) utilizando Python puro e a biblioteca **NLTK**.
- Refletir sobre as limitações dos modelos baseados em frequência (Problema da Esparsidade).

Parte 1: Contextualização

Abra qualquer aplicativo de mensagens no seu celular, digite a palavra "O" e clique na sugestão do meio do teclado repetidamente. Você verá frases inteiras se formando sozinhas.

Como o seu celular sabe o que você quer dizer? A resposta não é mágica, nem compreensão profunda do idioma. É pura contagem de repetições. O teclado aprendeu seus padrões de digitação. Se você sempre digita "*Bom*", ele sugere "*dia*" porque, no seu histórico estatístico, "*dia*" é a palavra com maior probabilidade de acompanhar "*Bom*".

Modelos Probabilísticos de Linguagem

Em vez de programar regras gramaticais complexas (sujeito, verbo, predicado) para o computador, nós entregamos um grande volume de textos (Corpus) e programamos a máquina para contar a coocorrência das palavras.

- **problema do histórico longo:** Tentar adivinhar a próxima palavra analisando todo o parágrafo anterior exige altíssimo poder computacional.

- **A solução N-gram (Suposição de Markov):** Nós limitamos a "*memória*" do algoritmo. Ele vai olhar apenas para a palavra atual (ou as poucas anteriores) para adivinhar o futuro. Pela Suposição de Markov, assume-se que o estado futuro depende apenas do estado presente (ou de um histórico imediato de tamanho fixo $N-1$).

Entendendo os N-grams

Um *N-gram* é simplesmente um recorte contíguo de N itens em um texto:

- **Unigrama ($N=1$):** Frequência de palavras soltas. Ex: "*gato*", "*dorme*".
- **Bigrama ($N=2$):** Analisa os pares consecutivos. Ex: "*a gatinha*", "*gatinha dorme*".
- **Trigrama ($N=3$):** Analisa trios consecutivos. Ex: "*a gatinha dorme*", "*gatinha dorme no*".

Parte 2: Extração de N-grams na Prática

Abra um novo Notebook no Google Colab. Copie e execute os blocos de código abaixo em células sequenciais para extrair *N-grams* de um texto de exemplo utilizando a biblioteca **NLTK** (Natural Language Toolkit).

Bloco 1: Instalação de Bibliotecas e Corpus

Instale/importe a biblioteca NLTK, baixe o tokenizador de sentenças e palavras e defina o corpus inicial de exemplo.

```
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

# Baixar recursos necessários para a tokenização
nltk.download('punkt_tab')

# Nosso Corpus inicial de exemplo
```

```
meu_texto = "A gatinha branca gosta de leite. A gatinha branca  
dorme no sofá."  
meu_texto = meu_texto.lower() # Padronizar para minúsculas  
  
# Tokenizar o texto em palavras individuais  
tokens = word_tokenize(meu_texto)  
print('Tokens extraídos:', tokens)
```

Bloco 2: Extraíndo N-Grams

Geramos os Bigramas e Trigramas a partir dos tokens extraídos e visualizamos as sequências formadas.

```
# Gerar bigramas (N=2) e trigramas (N=3)  
bigramas_extraídos = list(ngrams(tokens, 2))  
trigramas_extraídos = list(ngrams(tokens, 3))  
  
print('=== Exemplos de Bigramas ===')  
print(bigramas_extraídos[:5])  
  
print('\n=== Exemplos de Trigramas ===')  
print(trigramas_extraídos[:5])
```

Teste: Altere o número 2 na função *ngrams* para 3 e depois para 4. Observe como a "*janela de contexto*" do algoritmo aumenta e quais novas combinações são capturadas.

Parte 3: Construindo um "Autocompletar"

Vamos fazer o computador aprender probabilidades contando a frequência dos bigramas. A lógica estatística fundamental: "*Se a palavra 'gatinha' aparece 10 vezes no nosso texto, e em 8 dessas vezes a palavra seguinte é 'branca', então há 80% de probabilidade de 'branca' ser a próxima palavra.*"

Bloco 3: Treinando o Modelo com Frequências

Usamos a biblioteca nativa do Python para mapear cada palavra às suas possíveis continuações e contar quantas vezes essa coocorrência ocorre.

```
from collections import defaultdict, Counter

# Criar um dicionário de frequências: palavra_atual ->
# {proxima_palavra: contagem}
modelo_frequencias = defaultdict(Counter)

for w1, w2 in bigramas_extraidos:
    modelo_frequencias[w1][w2] += 1

print('=== Frequências coletadas para "gatinha" ===')
print(dict(modelo_frequencias['gatinha']))
```

Bloco 4: Consultando o Modelo

Criamos uma função que recebe uma palavra inserida pelo usuário, localiza as possíveis continuações no dicionário e as ordena pelas probabilidades mais altas.

```
def sugerir_proxima_palavra(palavra_digitada, modelo):
    palavra = palavra_digitada.lower()
    if palavra in modelo:
        # Encontrar as 3 continuações mais comuns
        sugestoes = modelo[palavra].most_common(3)
        # Calcular a porcentagem com base no total de ocorrências
        da palavra
        total_ocorrencias = sum(modelo[palavra].values())
        resultado = []
        for prox, count in sugestoes:
            prob = (count / total_ocorrencias) * 100
```

```

        resultado.append((prox, f"{prob:.1f}%"))
    return resultado
else:
    return []

# Executando a consulta
palavra_digitada = "gatinha"
sugestoes = sugerir_proxima_palavra(palavra_digitada,
                                     modelo_frequencias)
print(f'Sugestões para "{palavra_digitada}":', sugestoes)

```

Teste: Altere a variável `palavra_digitada` para *"gosta"* ou *"a"*. Execute a célula e veja como as probabilidades se ajustam com base nos padrões estatísticos do texto de treino.

Parte 4: Limitações e Desafio Prático

O Problema da Esparsidade

Se você digitar *"cachorro"* no modelo do Bloco 4, ele preverá *"gosta"* com sucesso por causa das regras do nosso pequeno corpus. No entanto, e se você digitar *"pássaro"*?

O código retornará uma **lista vazia**, ou seja, nenhuma sugestão. Isso ocorre porque *"pássaro"* tem **probabilidade empírica zero no modelo**, pois nunca apareceu no texto de treinamento. Isso se chama **Problema da Esparsidade** (Sparsity): os modelos probabilísticos clássicos são estritamente limitados ao vocabulário exato que lhes foi apresentado no corpus de treino. **Novas palavras quebram as previsões por completo.**

Atividade Laboratório

Trabalhando em seu Google Colab, resolva os seguintes pontos de forma individual, aplicando as modificações necessárias no código:

1. **Expansão do Corpus:** Modifique o texto do Bloco 1 (variável `meu_texto`). Acesse um portal de notícias de sua preferência, copie os três primeiros parágrafos de uma matéria de capa importante e cole na variável. Reexecute o treinamento (Blocos 1, 2, 3 e 4).
2. **Análise de Contexto:** Execute a consulta do Bloco 4 utilizando preposições extremamente comuns do nosso idioma, como por exemplo: "*de*", "*em*" ou "*com*". Anote as sugestões geradas pelo algoritmo.
3. **Avaliação Sintática:** Quais foram as 3 palavras mais sugeridas pelo seu modelo expandido para cada uma das preposições? Elas fazem sentido sintático? Discuta por escrito como o aumento do volume de dados afetou a "*inteligência*" das previsões se comparado ao mini-texto de exemplo original.

Entrega do Laboratório: Salve seu notebook com as respostas e o código executado e faça o download no formato `.ipynb`. Envie o arquivo através do sistema de entregas da disciplina até o final do prazo estabelecido.